

Análisis y desarrollo de software.

ID:314 5636

Instruct

or
Ing. Diego Fernando Calderon Silva
Correo: dfcalderon@sena.edu.co

JavaScr ipt

JavaScript

Es uno de los lenguajes de programación más populares, debido a que gracias a su escritura y gran comunidad, puedes ser un desarrollador:

- BackEnd
- FrontEnd
- Mobile

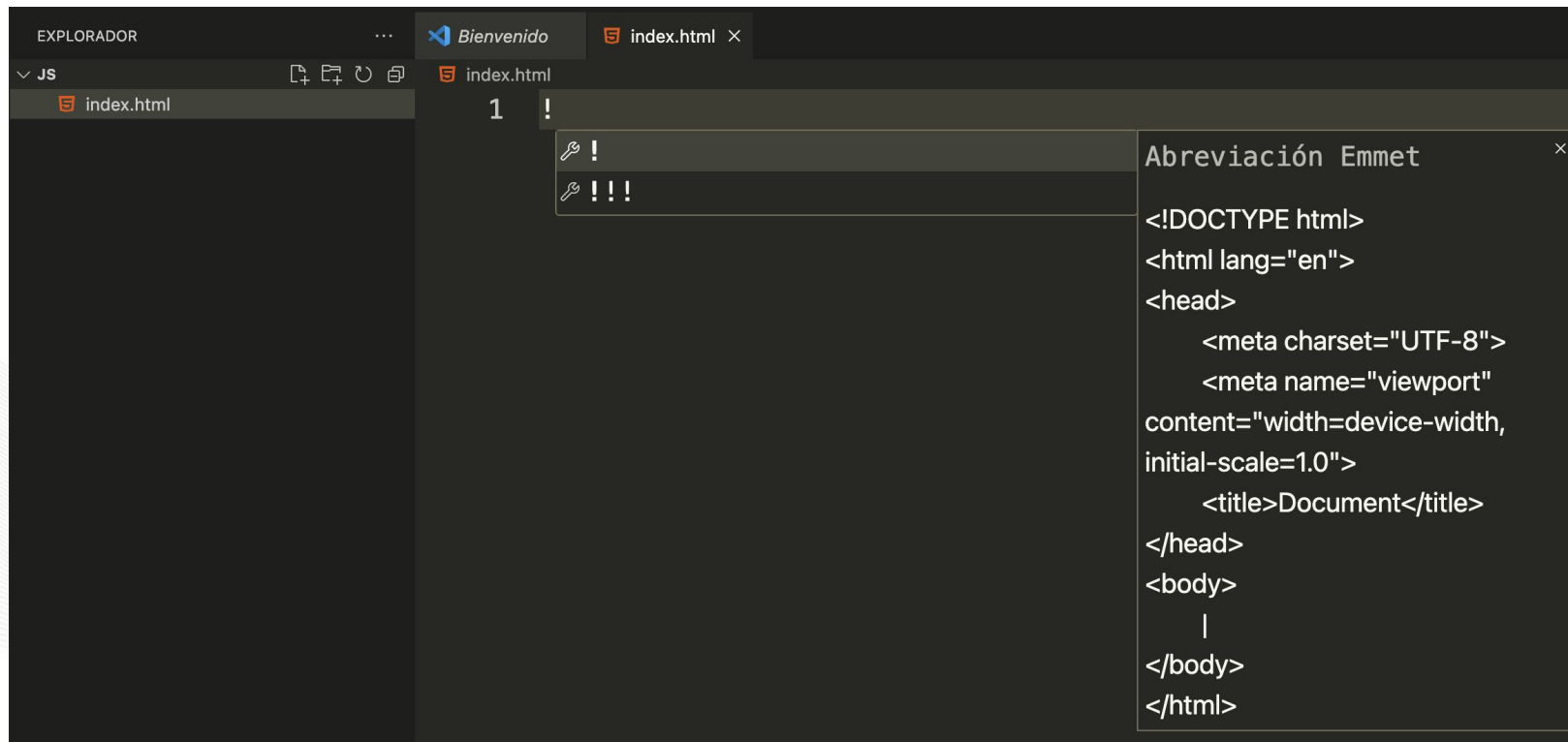
Y también esta implícita en las librerías o frameworks mas utilizados tales como:

- Vue
- Angular
- React

Preparando el ambiente de trabajo



Vamos a dedicar un tiempo para crear un entorno de trabajo limpio que nos ayude a trabajar más fácil, así que inicialmente vamos a crear un proyecto (directorio) en nuestro lugar de preferencia. Luego lo abriremos en VSC y escribiremos un archivo html, ya que es donde se va a aplicar la lógica de negocio; por lo tanto, hago uso de Emmet y escribo un signo de interrogación.



Preparando el ambiente de trabajo



Una vez creada la plantilla, vamos a escribir dentro del body un hola mundo, simplemente para verificar que todo este bien

A screenshot of a code editor interface. On the left, there's a sidebar with a file explorer showing a folder 'JS' containing a file 'index.html'. The main editor area shows the content of 'index.html'. The code is as follows:

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>JS</title>
7 </head>
8 <body>
9   Hola mundo
10 </body>
11 </html>
```

Preparando el ambiente de trabajo



Para poder usar código JS dentro del html, debemos apoyarnos de la etiqueta script, por lo tanto, haremos un hola mundo que se muestre en consola con lo siguiente:

```
<body>
  Hola mundo
  <script>
    console.log('Hola mundo');
  </script>
</body>
```

Abrimos el inspector del navegador, y vamos a la consola.

Consola borrada a la(s) 2:46:51 p.m.

Hola mundo

Código global – index.html:11

Preparando el ambiente de trabajo



¿Por qué el código js no se pone en la cabecera?



Preparando el ambiente de trabajo



Lo que acabamos de ver es un código básico de js que solo sirve para mostrar, pero en el mundo real de producción esto no es así, realmente se pueden escribir miles de líneas de código JS, por lo que debemos aprender a escribir nuestro código js en un solo archivo que este dentro de nuestro proyecto y luego referenciarlo en el html así:

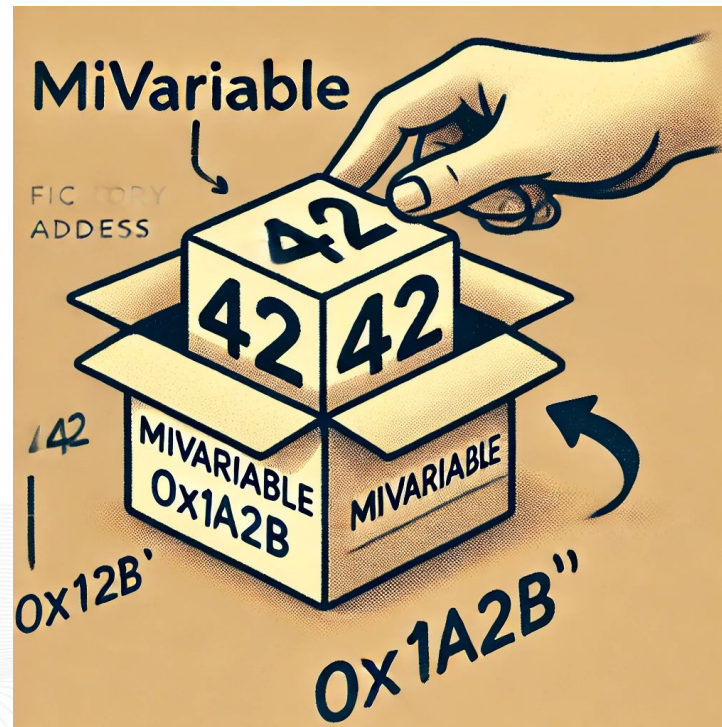
A screenshot of the Visual Studio Code interface. The Explorer sidebar on the left shows a project structure with a 'JS' folder containing 'app.js' and an 'index.html' file. The main editor area shows the 'app.js' file open, with a single line of code: `console.log('Hello World!');` on line 1.

```
<body>
  Hola mundo
  <script src="app.js"></script>
</body>
```

Variables



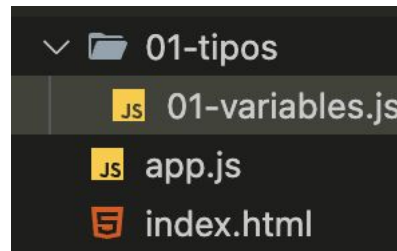
En JS y en cualquier lenguaje de programación existe un concepto llamado variable, el cual lo podemos ver como una caja en la cual podemos guardar datos cambiantes, esta caja siempre tendrá una dirección en memoria, por lo que cada que la usemos haremos referencia a este espacio en memoria, mas no a su contenido.



Variables



Para trabajar un poco mas cómodos, vamos a crear una carpeta llamada tipos, y dentro crearemos un script llamado variables.js asi:



Esto implica cambiar la referencia en index.html

```
<body>
  Hola mundo
  <script src="./01-tipos/01-variables.js"></script>
</body>
```

Recuerda siempre en lo posible trabajar con rutas relativas.

Variables

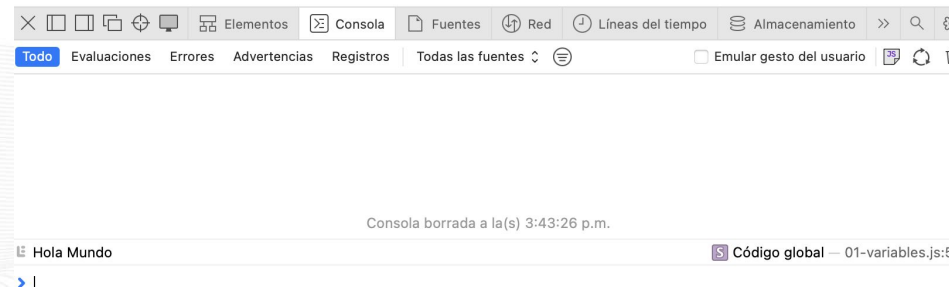


Ahora, vamos a crear una variable con la palabra reservada “let” seguido del nombre de la variable y luego de un igual, le voy a asignar el valor y cerramos con “;”

A screenshot of the Visual Studio Code editor interface. The Explorer panel on the left shows a project structure with a folder named '01-tipos' containing files '01-variables.js', 'app.js', and 'index.html'. The main editor window displays the content of '01-variables.js' with the following code:

```
1 // declaramos la variable
2 let nombreVariable = "Hola Mundo";
3
4 //mostramos la variable en pantalla
5 console.log(nombreVariable);
```

Hola mundo



Variables

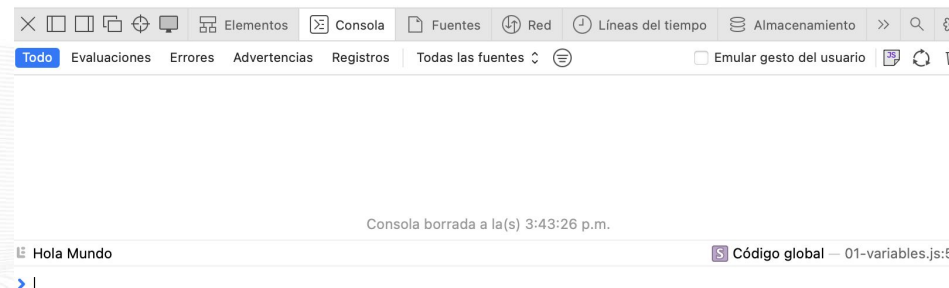


Ahora, vamos a crear una variable con la palabra reservada “let” seguido del nombre de la variable y luego de un igual, le voy a asignar el valor y cerramos con “;”

A screenshot of the Visual Studio Code editor interface. The Explorer panel on the left shows a project structure with a folder named "01-tipos" containing files "01-variables.js", "app.js", and "index.html". The main editor area shows the content of "01-variables.js" with the following code:

```
1 // declaramos la variable
2 let nombreVariable = "Hola Mundo";
3
4 //mostramos la variable en pantalla
5 console.log(nombreVariable);
```

Hola mundo



Reglas para declarar variables



El lenguaje como tal nos indica que hay formas de declarar una variable:

- Empiezan con letras
- No debe ser el mismo nombre de una palabra reservada
- Seguir la convención del UpperCamelCase o camelCase
- JS es case sensitive

```
let nombreVariable2;  
let NombreVariable3;
```

Reglas para declarar variables



Una vez definida la variable, puedo cambiar o darle valor luego, simplemente llamando únicamente la forma como la declaramos, es decir, sin la palabra `let` así:

```
let nombreVariable2;  
let NombreVariable3;  
  
nombreVariable2 = "Hola Mundo 2";
```

También, puedo declarar varios nombres de variables en una sola línea así:

```
// declaramos la variable  
let nombreVariable = "Hola Mundo";  
let nombreVariable2, nombreVariable3;  
nombreVariable2 = "Hola Mundo 2";  
nombreVariable3 = "Hola Mundo 3";
```

Tipos de datos



Tenemos dos tipos de datos:

Primitivos	Referencia
Number	Array
String	Object
Boolean	Function
Undefined	Class
Null	

Tipos primitivos



Lo primero que debemos recordar es que los datos primitivos los encontramos en la primera parte de la memoria, contrario a los de referencia que están en la parte dinámica. Empecemos por crear un script llamado 02-primitivos.js, asignamos una variable con un numero y otra con otro texto, y pongo el mouse arriba del nombre de la variable.

A screenshot of the Visual Studio Code editor interface. The Explorer panel on the left shows a file named '02-primitivos.js' selected under a folder named '01-tipos'. The main editor window displays the following JavaScript code:

```
1 let numero = 1;  
2 let string = "Hola Mundo";
```

A tooltip is visible over the word 'numero' on line 1, displaying the text 'let numero: number'.

Hagamos lo mismo para los demás tipos de dato.

A screenshot of the Visual Studio Code editor showing the same file '02-primitivos.js' with additional variable declarations:

```
1 let numero = 1;  
2 let string = "Hola Mundo";  
3 let verdadero = true;  
4 let falso = false;  
5 let dato;  
6 let indefinido = undefined;
```

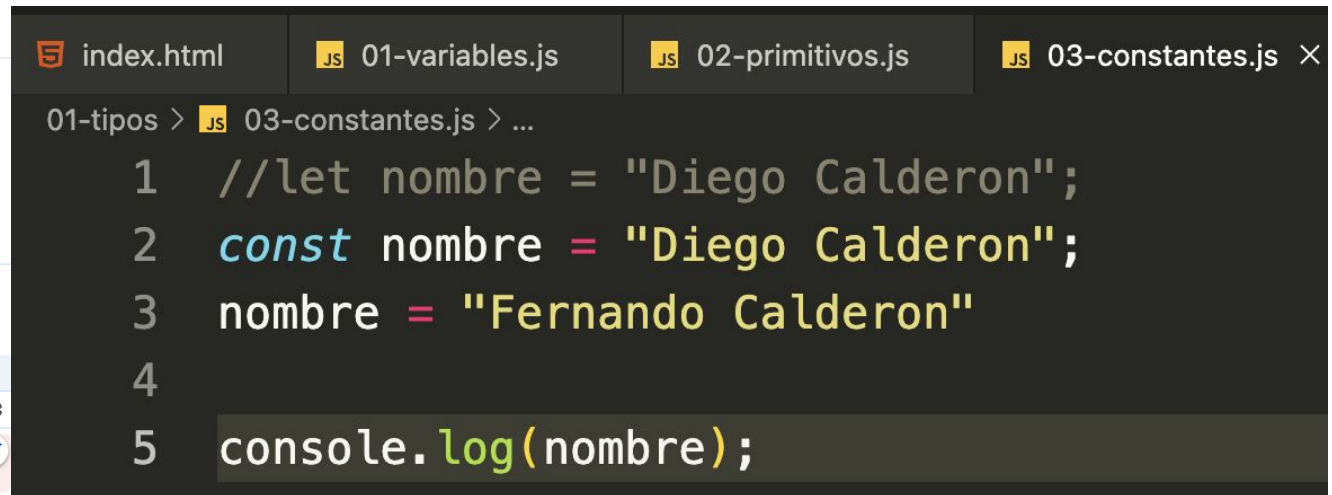
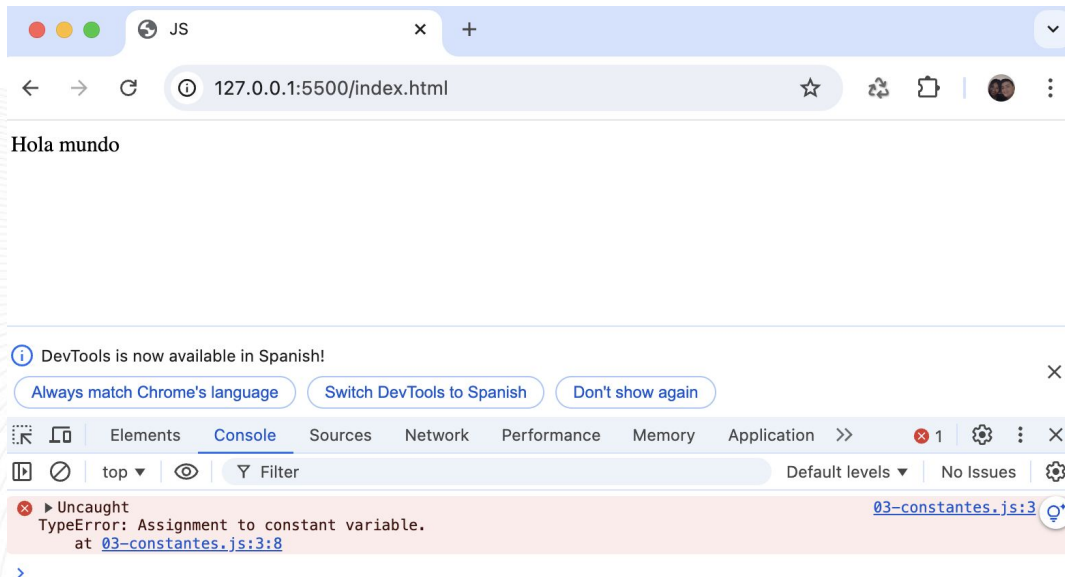
Constantes



Js, nos permite en el curso del trabajo darle valor a una variable y cambiarlo luego

```
let nombre = "Diego Calderon";  
nombre = "Fernando Calderon"  
console.log(nombre);
```

Por lo que al declarar con la palabra reservada const, eso debería cambiar.



Tipado dinámico



Dentro del mundo de la programación existen dos formas de tipar, una de ellas, el estático nos obliga a escribir que tipo de dato se va a usar seguido de la declaración de este, mientras que el dinámico, me dice que declare el dato y lo use que el se encargara de decirnos que tipo de dato es:

Estatico	Dinamico
String nombre = "Hola";	Nombre = "hola";
Java	JavaScript

Creemos un script llamado 04-tipado-dinamico.js y revisemos el tipo de dato

Tipado dinámico

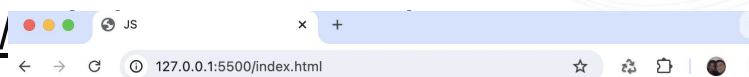


Creemos las siguientes variables y las mostramos así:

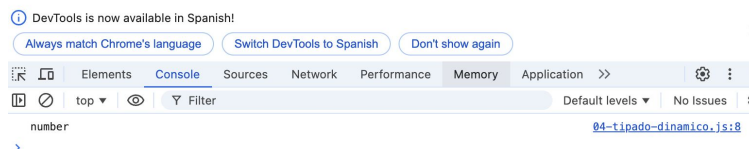
```
01-tipos > JS 04-tipado-dinamico.js > ...
1 let numero = 1;
2 let nombre = "Hola Mundo";
3 let verdadero = true;
4 let undef;
5 let nulo = null;
6
7 nombre = 53;
8 console.log(typeof nombre);
```

```
01-tipos > JS 04-tipado-dinamico.js > ...
1 let numero = 1;
2 let nombre = "Hola Mundo";
3 let verdadero = true;
4 let undef;
5 let nulo = null;
6
7 // nombre = 53;
8 console.log(typeof numero);
9 console.log(typeof nombre);
10 console.log(typeof verdadero);
11 console.log(typeof undef);
12 console.log(typeof nulo);
```

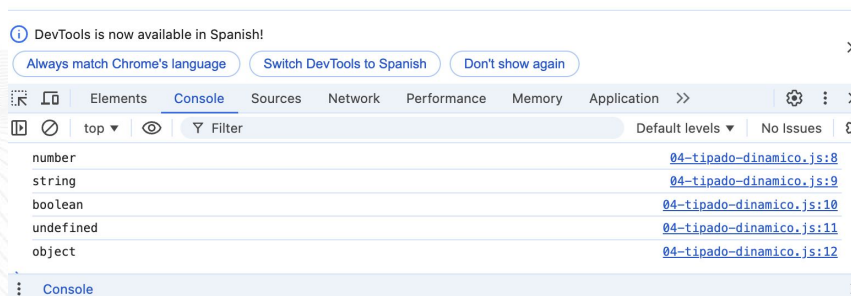
JS/docs/V



Hola mundo



Hola mundo



Objetos



Los objetos en js son un tipo de referencia, eso quiere decir que lo que realmente se guarda en la memoria es una referencia en memoria, lo que significa que son de tipo referencia y no primitivo. Son una agrupación de datos que tienen sentido al estar juntos y se declaran así:

```
01-tipos > JS 06-objetos.js > [e] apellido
1  let nombre = "Diego";
2  let apellido = "Calderon";
3  ⚡
4  let objetoPersona = {}
5
6  console.log(objetoPersona);
```

Si observamos en la consola, nos encontramos con que es un tipo objeto, y que esta vacío, así que vamos a llenarlo.

Objetos



Algo a tener en cuenta es que, al declarar propiedades dentro del objeto, se debe asignar su valor con dos puntos “:”

```
01-tipos > JS 06-objetos.js > [e] objetoPersona
1  let nombre = "Diego";
2  let apellido = "Calderon";
3
4  let objetoPersona = {
5      nombre: nombre,
6      apellido: "Calderon",
7      edad: 34,
8  }
9
10 console.log(objetoPersona);
```



Si nos fijamos, js nos asegura que los datos se muestran, pero no en el orden que nosotros los ingresamos.

Objetos



Ahora, para acceder a una propiedad, lo que debemos hacer es escribir el nombre del objeto.propiedadAl llamar así:

```
console.log(objetoPersona);  
console.log("Edad: ",objetoPersona.edad);
```



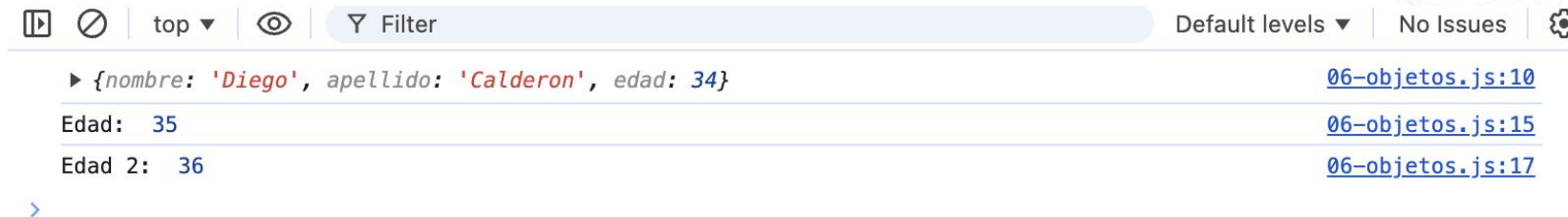
Otra forma es diciéndole al objeto la posición que queremos ver

```
console.log("Edad 2: ", objetoPersona["edad"]);
```

Objetos



Ahora, para poder darle valor por fuera del objeto a una propiedad, lo podemos hacer así:



```
objetoPersona.edad = 35;  
console.log("Edad: ",objetoPersona.edad);  
objetoPersona["edad"] = 36;  
console.log("Edad 2: ", objetoPersona["edad"]);
```

Finalmente, si lo que quiero es eliminar una propiedad lo puedo hacer así:

```
delete objetoPersona.apellido;  
console.log(objetoPersona);
```

Array



Un array de alguna forma es un arreglo de datos, no es tan denso como el objeto, pero sirve para almacenar varios datos, dándole una llave o posición dentro de el a cada uno de los diferentes tipos de dato. Su declaración es la siguiente

```
01-tipos > JS 07-array.js > ...
```

```
1 let array = ["Diego", "Calderon", 34];
```

```
01-tipos > JS 07-array.js > ...
```

```
1 let array = ["Diego", "Calderon", 34];
```

```
2 console.log(array);
```

```
3 console.log(array[0]);
```

```
4 console.log(array['Diego']);
```

Funciones.



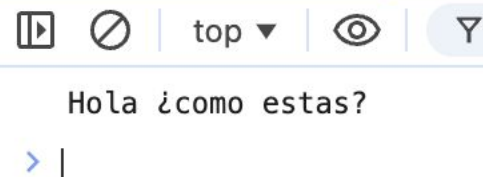
Son bloques fundamentales dentro del desarrollo de JS, podemos pensar en las funciones como procedimientos o pasos a seguir para cumplir un objetivo, pero también debemos ser claros en que las funciones se crean para una única acción. La forma de declarar y trabajarlas es la siguiente:

Declaración:

```
function saludar(){  
    console.log('Hola ¿como estas?');  
}
```

Llamado:

```
saludar(); // Llamado a la funcion
```



Funciones.



Creemos una calculadora usando funciones.



Declaraciones y expresiones.



Ahora que ya conocemos los tipos básicos de trabajo y datos, pasemos a ver expresiones y declaraciones ya que cada lenguaje tiene su forma propia de hacerlo.

Las declaraciones son aquellas que tiene de alguna forma una palabra reservada tales como:

Let, const, function, function*, class, export/import

Mientras que las expresiones son básicamente cuando se usan.

Una forma muy simple de identificar cual es el tipo, es poniendo un `console.log`, si el `console.log` les permite ver el dato es una expresión, de lo contrario es una declaración.

Operadores



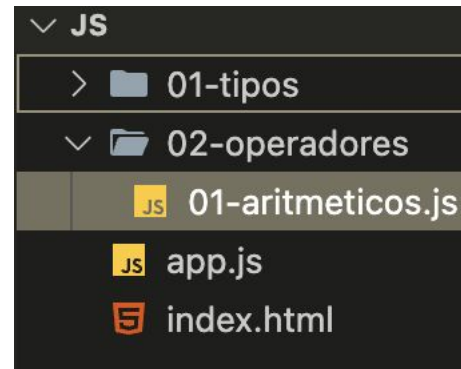
Los operadores nos sirven para crear expresiones que crean algoritmos y con estos solucionamos problemas, entre los operadores tenemos:

- Aritméticos
- Asignación
- Comparación
- Lógicos
- BitWise = 011001010

Operadores aritméticos



Los operadores aritméticos en JavaScript se utilizan para realizar operaciones matemáticas sobre números. Empecemos por crear un nuevo directorio así:



Para luego declarar los diferentes operadores:

```
1 let num1=5;
2 let num2=2;
3
4 console.log(num1+num2); // 7
5 console.log(num1-num2); // 3
6 console.log(num1*num2); // 10
7 console.log(num1/num2); // 2.5
8 console.log(num1%num2); // 1
```

Operadores de asignación



Los operadores de asignación en JavaScript se utilizan para asignar valores a las variables. Además de la asignación simple, existen operadores de asignación combinados que realizan una operación y asignan el resultado a la variable en una sola expresión.

```
1  let x = 10;  
2  x *= 5; // x es ahora 50
```

Operadores de comparación



Los operadores de comparación en JavaScript se utilizan para comparar dos valores y devuelven un valor booleano (true o false) basado en la comparación

```
1  let a = 5;
2  let b = "5";
3  let c = 10;
4
5  // Igualdad
6  console.log(a == b); // true (comparación después de conversión de tipo)
7
8  // Igualdad estricta
9  console.log(a === b); // false (diferente tipo de dato)
10
11 // Desigualdad
12 console.log(a != b); // false (comparación después de conversión de tipo)
13
14 // Desigualdad estricta
15 console.log(a !== b); // true (diferente tipo de dato)
16
17 // Mayor que
18 console.log(c > a); // true
19
20 // Mayor o igual que
21 console.log(c >= a); // true
22 console.log(a >= 5); // true
23
24 // Menor que
25 console.log(a < c); // true
26
27 // Menor o igual que
28 console.log(a <= c); // true
29 console.log(a <= 5); // true
30
```

Operadores lógicos



Los operadores lógicos en JavaScript se utilizan para combinar expresiones booleanas y devolver un valor booleano como resultado.

Los operadores lógicos tienen una precedencia que determina el orden en el que se evalúan las expresiones:

1. **NOT lógico (!)** tiene la precedencia más alta.
2. **AND lógico (&&)** tiene una precedencia intermedia.
3. **OR lógico (||)** tiene la precedencia más baja.

```
let a = true;
let b = false;
let c = true;

console.log(a || b && c); /
console.log((a || b) && c);
```

ShortCircuit



El **short-circuit** (o cortocircuito) en JavaScript es una característica de los operadores lógicos && (AND) y || (OR) que permite detener la evaluación de una expresión tan pronto como se determina su valor final. Esto puede mejorar el rendimiento y evitar la ejecución innecesaria de código.

Comportamiento del Short-Circuit

1. Operador OR Lógico (||):

1. Devuelve el valor del primer operando verdadero que encuentra. Si el primer operando es verdadero, no evalúa el segundo.

```
let a = true || false; // a es true (no se evalúa false)
let b = false || true; // b es true
let c = false || false; // c es false
```

ShortCircuit



Operador AND Lógico (&&):

- Devuelve el valor del primer operando falso que encuentra. Si el primer operando es falso, no evalúa el segundo.

```
let a = true && false; // a es false
let b = true && true;  // b es true
let c = false && true; // c es false (no se evalúa true)
```

ShortCircuit



Ejemplos de Uso del Short-Circuit OR Lógico (||)

El operador OR se usa comúnmente para establecer valores predeterminados.

```
function obtenerNombre(usuario) {  
    return usuario.nombre || "Anónimo";  
}  
  
let usuario1 = { nombre: "Carlos" };  
let usuario2 = {};  
  
console.log(obtenerNombre(usuario1)); // "Carlos"  
console.log(obtenerNombre(usuario2)); // "Anónimo"
```

ShortCircuit



AND Lógico (&&)

El operador AND se usa comúnmente para ejecutar código solo si se cumplen ciertas condiciones.

```
let usuario = {  
  nombre: "Ana",  
  edad: 29  
};  
  
usuario && usuario.edad >= 18 && console.log("Eres mayor de edad");
```

Orden de operaciones



Ejecuta la siguiente lógica

```
console.log(8/2*(2+2));
```

Operador ternario



El operador ternario en JavaScript es un operador condicional que permite evaluar una expresión y devolver un valor basado en esa evaluación, todo en una sola línea de código. Es una forma compacta y eficiente de realizar una operación condicional. Su sintaxis es:

```
condición ? expresiónSiVerdadero : expresiónSiFalso;
```

Caso de uso

```
index.html JS 08-ternario.js x
02-operadores > JS 08-ternario.js > ...
1 let edad = 18;
2 let mensaje = (edad >= 18) ? "Eres mayor de edad" : "Eres menor de edad";
3 console.log(mensaje); // "Eres mayor de edad"
```

Instrucción if



El if en JavaScript es una estructura de control condicional que permite ejecutar un bloque de código solo si se cumple una condición especificada. Es fundamental para la toma de decisiones en la programación.

Vamos a crear un directorio llamado 03-controlFlujo y dentro el primer script 01-if.js

```
if (condición) {  
    // Bloque de código que se ejecuta si la condición es verdadera  
}
```

```
let edad = 18;  
  
if (edad >= 18) {  
    console.log("Eres mayor de edad");  
}
```

Instrucción while



El while en JavaScript es una estructura de control de bucle que permite ejecutar un bloque de código repetidamente mientras se cumpla una condición especificada. Es útil cuando no se sabe de antemano cuántas veces se necesita repetir un bloque de código.

```
while (condición) {  
    // Bloque de código que se ejecuta mientras la condición sea verdadera  
}
```

```
let contador = 0;  
  
while (contador < 5) {  
    console.log("El contador es: " + contador);  
    contador++;  
}
```

```
El contador es: 0  
El contador es: 1  
El contador es: 2  
El contador es: 3  
El contador es: 4
```

```
> |
```

Bucle infinito



Si la condición del while siempre es true, se crea un bucle infinito, lo cual puede congelar tu programa o navegador. Asegúrate de tener una condición de salida adecuada.

```
while (true) {  
    console.log("Esto es un bucle infinito");  
    // Debes tener una forma de salir del bucle, como un break en ciertas condiciones  
}
```

Do while



El bucle do...while en JavaScript es similar al bucle while, pero con una diferencia clave: el bloque de código dentro del do...while se ejecuta al menos una vez antes de que se evalúe la condición. Después de la primera ejecución, la condición se evalúa y, si es verdadera, el bucle continúa ejecutándose. Si es falsa, el bucle termina

```
do {  
    // Bloque de código que se ejecuta al menos una vez  
} while (condición);
```

```
let contador = 0;  
  
do {  
    console.log("El contador es: " + contador);  
    contador++;  
} while (contador < 5);
```

```
El contador es: 0  
El contador es: 1  
El contador es: 2  
El contador es: 3  
El contador es: 4
```

Do while



Diferencia entre while y do...while

La diferencia principal entre while y do...while es el punto en el que se evalúa la condición:

- En un bucle while, la condición se evalúa antes de que se ejecute el bloque de código.
- En un bucle do...while, el bloque de código se ejecuta al menos una vez antes de que se evalúe la condición.

```
let contadorWhile = 0;

while (contadorWhile < 0) {
  console.log("Este mensaje no se mostrará en 'while'");
}

let contadorDoWhile = 0;

do {
  console.log("Este mensaje se mostrará una vez en 'do...while'");
} while (contadorDoWhile < 0);
```

Este mensaje se mostrará una vez en 'do...while'

For



El ciclo for en JavaScript es una estructura de control que permite repetir un bloque de código un número específico de veces. Es particularmente útil cuando se conoce de antemano cuántas veces se debe repetir la ejecución del bloque de código.

```
for (inicialización; condición; actualización) {  
    // Bloque de código que se ejecuta mientras la condición sea verdadera  
}
```

```
for (let i = 0; i < 5; i++) {  
    console.log("El valor de i es: " + i);  
} ✨
```

El valor de i es: 0

El valor de i es: 1

El valor de i es: 2

El valor de i es: 3

El valor de i es: 4

For



También lo podemos aplicar en arrays

```
let frutas = ["manzana", "banana", "cereza"];

for (let i = 0; i < frutas.length; i++) {
  console.log("Fruta: " + frutas[i]);
} ✨
```

Fruta: manzana

Fruta: banana

Fruta: cereza

For anidado



El caso más usado es el for anidado.

```
let matriz = [  
  [1, 2, 3],  
  [4, 5, 6],  
  [7, 8, 9]  
];  
  
for (let i = 0; i < matriz.length; i++) {  
  for (let j = 0; j < matriz[i].length; j++) {  
    console.log("Elemento en [" + i + "][" + j + "] es: " + matriz[i][j]);  
  }  
}
```

Elemento en [1][0] es: 4
Elemento en [1][1] es: 5
Elemento en [1][2] es: 6
Elemento en [2][0] es: 7
Elemento en [2][1] es: 8
Elemento en [2][2] es: 9

ForOf



for...of para Iterar sobre Iterables

El bucle for...of es una forma moderna y concisa de iterar sobre los elementos de un iterable (arrays, strings, mapas, sets, etc.).

```
let frutas = ["manzana", "banana", "cereza"];

for (let fruta of frutas) {
  console.log("Fruta: " + fruta);
}
```

Fruta: manzana

Fruta: banana

Fruta: cereza

ForIn



for...in para Iterar sobre Propiedades de Objetos

El bucle for...in se utiliza para iterar sobre las propiedades enumerables de un objeto.

```
let persona = {  
  nombre: "Juan",  
  edad: 30,  
  ciudad: "Bogotá"  
};  
  
for (let propiedad in persona) {  
  console.log(propiedad + ": " + persona[propiedad]);  
}
```

```
nombre: Juan  
edad: 30  
ciudad: Bogotá
```

Switch



El switch en JavaScript es una estructura de control que permite ejecutar diferentes bloques de código basados en el valor de una expresión. Es útil cuando necesitas evaluar una variable o expresión contra múltiples posibles valores y ejecutar diferentes bloques de código según el caso

```
switch (expresión) {  
  case valor1:  
    // Bloque de código que se ejecuta si expresión === valor1  
    break;  
  case valor2:  
    // Bloque de código que se ejecuta si expresión === valor2  
    break;  
  // Puedes agregar tantos casos como necesites  
  default:  
    // Bloque de código que se ejecuta si ninguno de los casos anteriores coincide  
}
```

Switch



Ejemplo básico.

```
let fruta = "manzana";

switch (fruta) {
  case "manzana":
    console.log("Es una manzana");
    break;
  case "banana":
    console.log("Es una banana");
    break;
  case "cereza":
    console.log("Es una cereza");
    break;
  default:
    console.log("Fruta desconocida");
}
```

Ejercicios



1. Construir una función que reciba dos parámetros, y la función debe devolver el número que sea mayor para mostrarlo en pantalla.
2. Construir una función que se llame nombreResolucion la cual va a recibir dos parámetros ancho y alto, esta función deberá devolvernos el nombre de la resolución dependiendo de los parámetros enviados, las resoluciones aceptadas son:

```
/**
 * Nombre: ancho x alto
 * 8K 7680 x 4320
 * 💡 4K 3840 x 2160
 * WQHD 2560 x 1440
 * FHD 1920 x 1080
 * HD 1280 x 720
 */
```

3. Construir una función llamada getByIdx que nos permita obtener un elemento basado en su índice, esta función recibirá dos parámetros, arr e idx. El valor que le pasemos como índice no puede ser menor que cero, además, el valor del elemento debe existir en el array

Ejercicios



1. Usando un loop, generar un algoritmo que me imprima la cantidad de números impares que existen del 0 al 10
2. Crear un algoritmo que me devuelva el número menor y también el número mayor de un array de 7 campos, para lograr esto, debe escribir una función llamada `getMenorMayor` que recibe un parámetro llamado `arr` y retornar lo solicitado inicialmente.
3. Crear un algoritmo que devuelva la cantidad de numeros positivos dentro de un array cualquiera. Para lograr esto, debe escribir una función `cuantosPositivos` que recibe un parrametro `arr`
4. Crear una función llamada `precioCompleto`, la cual deberá funcionar como una calculadora de impuestos, por lo tanto, deberá recibir un valor flotante en su primer parámetro `precio` y un valor flotante en su segundo parámetro `impuesto`. Deberá retornar el resultado de la operación luego de calcular el valor final del impuesto a pagar.
5. Crear una función llamada `toPair`, la cual tomará un array de objetos y devolverá un array de pares



GRACIAS

Línea de atención al ciudadano: 01 8000 910270
Línea de atención al empresario: 01 8000 910682



www.sena.edu.co